An Analysis of Adagrad optimizer

Adagrad optimizer

Adagrad optimizer -- Part 1

G j , j = ∑ τ = 1 t g τ , j 2 . {\displaystyle G_{j,j}=\sum _{\tau =1}^{t}g_{\tau ,j}^{2}.} This vector essentially stores a historical sum of gradient squares by dimension and is updated after every iteration. The formula for an update is now

Adagrad optimizer -- Part 2

Adam Adam (short for Adaptive Moment Estimation) is a 2014 update to the RMSProp optimizer combining it with the main feature of the Momentum method. In this optimization algorithm, running averages with exponential forgetting of both the gradients and the second moments of the gradients are used. Given parameters w (t) {\displaystyle w^{(t)}} and a loss function L (t) {\displaystyle L^{(t)}} , where t {\displaystyle t} indexes the current training iteration (indexed at 1 {\displaystyle 1}), Adam's parameter update is given by:

Adagrad optimizer -- Part 3

RMSProp RMSProp (for Root Mean Square Propagation) is a method invented in 2012 by James Martens and Ilya Sutskever, at the time both PhD students in Geoffrey Hinton's group, in which the learning rate is, like in Adagrad, adapted for each of the parameters. The idea is to divide the learning rate for a weight by a running average of the magnitudes of recent gradients for that weight. Unusually, it was not published in an article but merely described in a Coursera lecture.

Adagrad optimizer -- Part 4

Notable applications Stochastic gradient descent is a popular algorithm for training a wide range of models in machine learning, including (linear) support vector machines, logistic regression (see, e.g., Vowpal Wabbit) and graphical models. When combined with the backpropagation algorithm, it is the de facto standard algorithm for training artificial neural networks. Its use has been also reported in the Geophysics community, specifically to applications of Full Waveform Inversion (FWI). Stochastic gradient descent competes with the L-BFGS algorithm, which is also widely used. Stochastic gradient descent has been used since at least 1960 for training linear regression models, originally under the name ADALINE. Another stochastic gradient descent algorithm is the least mean squares (LMS) adaptive filter.

Adagrad optimizer -- Part 5

History In 1951, Herbert Robbins and Sutton Monro introduced the earliest stochastic approximation methods, preceding stochastic gradient descent. Building on this work one year later, Jack Kiefer and Jacob Wolfowitz published an optimization algorithm very close to stochastic gradient descent, using central differences as an approximation of the gradient. Later in the 1950s, Frank Rosenblatt used SGD to optimize his perceptron model, demonstrating the first applicability of stochastic gradient descent to neural networks. Backpropagation was first described in 1986, with stochastic gradient descent being used to efficiently optimize parameters across neural networks with multiple hidden layers. Soon after, another improvement was developed: mini-batch gradient descent, where small batches of data are substituted for single samples. In 1997, the practical performance benefits from vectorization achievable with such small batches were first explored, paving the way for efficient optimization in machine learning. As of 2023, this mini-batch approach remains the norm for training neural networks, balancing the benefits of stochastic gradient descent with gradient descent. By the 1980s, momentum had already been introduced, and was added to SGD optimization techniques in 1986. However, these optimization techniques assumed constant hyperparameters, i.e. a fixed learning rate and momentum parameter. In the 2010s, adaptive approaches to applying SGD with a per-parameter learning rate were introduced with AdaGrad (for "Adaptive Gradient") in 2011 and RMSprop (for "Root Mean Square Propagation") in 2012. In 2014, Adam (for "Adaptive Moment Estimation") was published, applying the adaptive approaches of RMSprop to momentum; many improvements and branches of Adam were then developed such as Adadelta, Adagrad, AdamW, and Adamax. Within machine learning, approaches to optimization in 2023 are dominated by Adam-derived optimizers, TensorFlow and PyTorch, by far the most popular machine learning libraries, as of 2023 largely only include Adam-derived optimizers, as well as predecessors to Adam such as RMSprop and classic SGD. PyTorch also partially supports limited-memory BFGS, a line-search method, but only for single-device setups without parameter groups.

Adagrad optimizer -- Part 6

where, γ {\displaystyle \gamma } is the forgetting factor. The concept of storing the historical gradient as sum of squares is borrowed from Adagrad, but "forgetting" is introduced to solve Adagrad's diminishing learning rates in non-convex problems by gradually decreasing the influence of old data. And the parameters are updated as,

Adagrad optimizer -- Part 7

[w 1 w 2] ← [w 1 w 2] − η [∂ ∂ w 1 (w 1 + w 2 x i − y i) 2 ∂ ∂ w 2 (w 1 + w 2 x i − y i) 2] = [w 1 w 2] − η [2 (w 1 + w 2 x i − y i) 2 x i (w 1 + w 2 x i − y i)] . {\displaystyle

{\begin{bmatrix}w_{1}\\w_{2}\end{bmatrix}}\leftarrow {\begin{bmatrix}w_{1}\\w_{2}\end{bmatrix}}-\eta {\begin{bmatrix}{\frac {\partial }{\partial w_{1}}}(w_{1}+w_{2}x_{i}-y_{i})^{2}\\{\frac {\partial }{\partial w_{2}}}(w_{1}+w_{2}x_{i}-y_{i})^{2}\end{bmatrix}}={\begin{bmatrix}w_{1}\\w_{2}\end{bmatrix}}-\eta {\begin{bmatrix}2(w_{1}+w_{2}x_{i}-y_{i})\\2x_{i}(w_{1}+w_{2}x_{i}-y_{i})\end{bmatrix}}.} Note that in each iteration or update step, the gradient is only evaluated at a single x i {\displaystyle x_{i}} . This is the key difference between stochastic gradient descent and batched gradient descent. In general, given a linear regression y ^ = ∑ k ∈ 1 : m w k x k {\displaystyle {\hat {y}}=\sum _{k\in 1:m}w_{k}x_{k}} problem, stochastic gradient descent behaves differently when m < n {\displaystyle m<n} (underparameterized) and m ≥ n {\displaystyle m\geq n} (overparameterized). In the overparameterized case, stochastic gradient descent converges to arg ⁡ min w : w T x k = y k ∀ k ∈ 1 : n ⁡ ‖ w − w 0 ‖ {\displaystyle \arg \min _{w:w^{T}x_{k}=y_{k}\forall k\in 1:n}\|w-w_{0}\|} . That is, SGD converges to the interpolation solution with minimum distance from the starting w 0 {\displaystyle w_{0}} . This is true even when the learning rate remains constant. In the underparameterized case, SGD does not converge if learning rate remains constant.

Adagrad optimizer -- Part 8

max ⁡ k = 0 , … , ⌊ T / η ⌋ | E [g (w k)] − E [g (W k η)] | ≤ C η 2 . {\displaystyle \max _{k=0,\dots ,\lfloor T/\eta \rfloor }\left|\mathbb {E} [g(w_{k})]-\mathbb {E} [g(W_{k\eta })]\right|\leq C\eta ^{2}.}